Hummingbird RL: the algorithms

From PID and LQR to PPO, TD3 and SAC, and what each part of them does

Nathan Conti Alunno

October 2026

Abstract. We train a simulated hummingbird to hover, reach points, avoid obstacles and follow a course. Hovering is unstable and the air is gusty, so we first set two classical controllers as the reference for stability: a cascaded proportional-integral-derivative controller (PID) and a linear-quadratic regulator (LQR). We then ask how far three deep reinforcement learning algorithms go beyond them: proximal policy optimisation (PPO), twin delayed deep deterministic policy gradient (TD3) and soft actor-critic (SAC). This report explains the algorithms: for each part, its equation, the line of the single-file CleanRL implementation that computes it, the Stable-Baselines3 (SB3) setting that controls it, and, where we ran one, the ablation that shows what it does on our bird. The full experimental results are in the W&B report; here they serve as evidence for how the algorithms work.

I chose this problem, defined the tasks and the experiments, ran all 124 training runs and the evaluations on my own computer, and interpreted the results. I used Claude (Anthropic) as a tool to write most of the code and to draft the text, which I reviewed and corrected. The end of the report says how the work was shared and what I learned.

Links: the results (W&B report) · interactive demo (the bird flies live in your browser, with every controller) · code on GitHub · every training and evaluation run (W&B project)

Key findings.

- *Classical control is hard to beat in calm air.* PID and LQR succeed in 100% of the hover and go-to-point tests. Under wind and wing damage, the PID, thanks to its integrators, stays the most robust controller on hover (robust area 0.82); on go-to-point, only agents trained in randomised conditions beat it (PPO 0.96 against 0.81).
- *Learning pays off where the classical controllers must plan.* Around obstacles SAC succeeds in 98% and PPO in 89% of the episodes, against 44 to 52% for the potential-field controllers. On the long navigation course, though, the cautious PID stays best (41%, against 18% for PPO and 27% with transfer).
- *A policy without memory is fragile, unless its training was not.* Trained in calm air, every learned policy loses to the PID under wind (robust area 0.08 to 0.39). Trained in randomised conditions, they match or beat it on go-to-point (0.82 to 0.96), and after a sudden wing failure PPO deviates by 1.5 cm against 9.1 cm for the PID.
- *PPO is the most reliable learner, SAC and TD3 the most sample-efficient.* The off-policy methods need up to three times fewer samples, but their success swings up and down during training, so the last model is a lottery (SAC: 64% at the end, 99% at its best checkpoint). A decaying learning rate does not fix it.
- *Some parts of PPO are essential here, others are not.* Without clipping PPO crashes, and with one-step advantages ($\lambda = 0$) it stays up but never comes back to its target: 0% success either way. Advantage normalisation changes nothing, with or without reward normalisation.
- *PPO's critic ranks near-hover states like the LQR's exact cost-to-go* (rank correlation 0.96), but PPO's flights look more like the PID's.

1. The problem as a Markov decision process

Transition. The bird is one rigid body: position, velocity, orientation (a quaternion) and angular rates. Its wings are averaged over each wingbeat. A wing produces a thrust proportional to the square of its speed, and the controller steers by changing the two amplitudes, the frequency and the tilts of the stroke planes. Physics runs at 240 Hz and the controller decides at 60 Hz, holding each action for four physics steps. Hovering is unstable: an uncontrolled bird develops an oscillation whose amplitude doubles in about 0.75 s. Wind (a gust process), sensor noise and actuator noise are always on.

Actions. An action is $a = (a_1, \dots, a_5) \in [-1, 1]^5$: the amplitudes of the left and right wings (a_1, a_2, from 0 to 80°; their difference rolls the bird), the wingbeat frequency (a_3, 32 to 48 Hz), the tilt of the stroke plane (a_4, ±20°, a pitch

torque) and the opposite tilts of the two stroke planes (a_5, $\pm 20°$, a yaw torque). Hovering needs about $a_1 = a_2 = 0.6$ ($64°$ of amplitude) and $a_3 = a_4 = a_5 = 0$.

Observations. The controller receives $o \in \mathbb{R}^{48}$: altitude, velocity, roll, pitch, the three body rates, the previous action, up to two targets and the five nearest obstacles. Everything is relative to the bird and expressed in its *heading frame* (the world turned with the bird's yaw), so a target 30 cm ahead looks the same whichever way the bird faces. The observation hides the wind, the mass and any wing damage. Strictly, the problem is therefore a partially observable Markov decision process (POMDP): the same o can come from a calm day or from a gusty one. This matters for the learned controllers (Section 3).

Rewards are bounded and mostly positive near the goal. If they were mostly negative, the quickest way to stop losing points would be to crash on purpose.

Task	Reward per decision r_t	End of episode
Hover & recover (5 s)	$e^{-d_t/5\,\mathrm{cm}} - 0.05\,(\mathrm{roll}^2 + \mathrm{pitch}^2) - 0.01\,\omega_{\mathrm{yaw}}^2$	success judged at the end: mean distance over the last 2 s below 3 cm
Go-to-point (5 s), Obstacles (6 s)	$100\,(d_{t-1} - d_t) + 0.5\,e^{-d_t/3\,\mathrm{cm}}$	+50 when the target is *held* (within 3 cm, slower than 10 cm/s, for 15 decisions)
Navigation (15 s, 3 waypoints)	$100\,(d_{t-1} - d_t) - 0.01$	+20 per waypoint, +50 when the last one is held

Here d_t is the distance in metres from the bird to its current target after decision t, so $100\,(d_{t-1} - d_t)$ pays one point per centimetre of progress; roll and pitch are in radians and ω_{yaw} is the yaw rate in rad/s. Every task adds $-0.02\,\|a_t - a_{t-1}\|^2 - 0.01\,P_t/P_{\mathrm{hover}}$, a price for jerky actions and for energy, where P_t is the aerodynamic power of the wings and P_{hover} the power needed to hover. A crash, leaving the arena, tilting past $80°$ or touching an obstacle ends the episode with -10.

Objective. Let s_t be the full state of the simulator at decision t. A policy $\pi(a \mid s)$ (the probability density of choosing a in s) is judged by its expected discounted return $J(\pi) = \mathbb{E}_\pi\big[\sum_{t\ge 0} \gamma^t r_t\big]$, where the expectation is over episodes and $\gamma = 0.99$. Rewards are discounted with a half-life of about 70 decisions (1.2 s), and the effective horizon $1/(1-\gamma)$ is 100 decisions. The value $V^\pi(s) = \mathbb{E}_\pi[\sum_{k\ge 0} \gamma^k r_{t+k} \mid s_t = s]$ is the return to expect from s, and the action value $Q^\pi(s,a)$ the same after first taking a. They satisfy the Bellman equation (Sutton & Barto 2018), where s' is the next state:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi,\, s'}\big[r(s,a) + \gamma\, V^\pi(s')\big].$$

Terminated vs truncated. An episode ends either because something happened (success or failure: *terminated*, the value of what follows is zero) or because time is up (*truncated*: the bird would have gone on flying). Treating a time limit as a termination teaches the agent that the world ends at 5 s (Pardo et al. 2018). SB3 keeps the two apart. For PPO it adds $\gamma V(s_T)$, the discounted value of the last state, to the last reward of a truncated episode. For TD3 and SAC its replay buffer remembers which transitions ended by time-out and treats them as ordinary transitions, whose target still includes the value of the next state (the setting `handle_timeout_termination`, on by default).

Why continuous control. The five actions are continuous and their effects are smooth. Discretising them (as a deep Q-network, DQN, would need) gives k^5 actions for k levels per action and throws away the smoothness. All three algorithms below output real-valued actions directly.

Notation. t counts decisions (1/60 s). θ always denotes the weights of a policy (the *actor*) and ψ those of a value network (the *critic*); a bar ($\bar{\psi}$) marks a slowly updated copy. A hat marks an estimate computed from data ($\widehat{A}_t$). $\mathbb{E}$ is an expectation, $\mathcal{H}$ an entropy, $\mathcal{N}(\mu, \sigma^2)$ a Gaussian.

Section 2 controls this bird with a model and no data; Sections 3 and 4 learn the controller from data, without a model.

2. Classical control

Cascaded PID. This is the structure of quadrotor autopilots: loops nested inside each other, each faster than the one around it. All signals come from the observation, in the heading frame.

1. *Position loop.* In: the position error e_p (target minus bird). Out: a desired velocity $v_{\text{des}} = K_p e_p$ ($K_p = 5\ \text{s}^{-1}$), capped at 0.6 m/s so that far targets are approached at a steady speed.
2. *Velocity loop.* In: the velocity error $v_{\text{des}} - v$. Out: a desired acceleration a_{des}, a proportional term plus an integral that only runs within 15 cm of the target, plus the push of the potential field near obstacles.
3. *Geometry.* In: a_{des}. Out: the thrust and attitude that produce it. A hovering bird can only push along its back, so to accelerate forward it must tilt nose down: $\text{pitch}_{\text{des}} = -\arctan\left(a_x/(g+a_y)\right)$, $\text{roll}_{\text{des}} = \arctan\left(a_z/(g+a_y)\right)$, both limited to 25°, and thrust $T = m\,(g+a_y)/(\cos\text{roll}\cos\text{pitch})$, where m is the mass and g gravity.
4. *Attitude loop.* In: the errors $\text{roll}_{\text{des}} - \text{roll}$, $\text{pitch}_{\text{des}} - \text{pitch}$ and the body rates. Out: roll and pitch torques from a PID on each angle (tuned as second-order responses of 30 and 20 rad/s), and a yaw torque that only damps the yaw rate.
5. *Allocation* (the "mixer"). In: T and the three torques. Out: the five actions, by inverting the wing model. The total thrust and the roll torque fix the two amplitudes ($F_L + F_R = T$, $(F_L - F_R)\,r_w = \tau_{\text{roll}}$, where r_w is the wing's lever arm), the pitch torque fixes the tilt, the yaw torque the opposite tilts, and the frequency stays at its nominal value unless a wing runs out of amplitude.

The two integrals make it robust: the velocity integral cancels a steady wind, and the attitude integral cancels the steady torque of a damaged wing. Since yaw is only damped, the PID never turns: it backs up or flies sideways to a target behind it.

Potential field (Khatib 1986). Each obstacle closer than $\ell_0 = 15$ cm (surface to surface) pushes the bird away along the gradient of $\frac{1}{2}k_{\text{rep}}\,(1/\ell - 1/\ell_0)^2$, where ℓ is the gap between the two surfaces. We add a sideways component so that the bird can slide around an obstacle that blocks the way. The PID adds this push to its desired acceleration, and the LQR uses it to move its target (a "carrot"). Fields have local minima where attraction and repulsion cancel (Koren & Borenstein 1991). This is one reason why the classical controllers do poorly on Obstacles (PID 44%, LQR 52% success; the PID mostly runs out of time in front of an obstacle) while PPO reaches 89% and SAC 98%.

LQR. Near hover the bird is almost linear: $x_{k+1} \approx Ax_k + Bu_k$, where k counts decisions. The state $x \in \mathbb{R}^{11}$ is the deviation from hovering still at the target (position error, velocity, roll, pitch, the three body rates, in the heading frame), and $u = a - a_{\text{hover}} \in \mathbb{R}^5$ the deviation from the hover action. So A is 11×11 and B is 11×5. We do not derive them by hand: we linearise the simulator itself, by central finite differences of the 60 Hz map. The open loop has an eigenvalue of 1.016 per step, which is the instability, and the pair (A, B) is controllable. The cost is $\sum_k x_k^\top Q x_k + u_k^\top R u_k$, where the diagonal matrices Q and R say how much each state deviation and each action costs. Its Bellman equation (a cost, so a minimum, and no discount) is

$$V(x) = \min_u \left[x^\top Q x + u^\top R u + V(Ax + Bu)\right].$$

Try $V(x) = x^\top P x$ with P symmetric. The minimum is reached at $u^* = -Kx$, with the 5×11 gain $K = (R + B^\top P B)^{-1} B^\top P A$. Substituting back gives the discrete algebraic Riccati equation

$$P = A^\top P A - A^\top P B\,(R + B^\top P B)^{-1} B^\top P A + Q.$$

So $x^\top P x$ is the *exact* value function of the linear-quadratic problem, and the gain is greedy with respect to it. This is the RL picture solved in closed form, because the model is known and linear. The weights follow Bryson's rule (Bryson & Ho 1975): $Q_{ii} = 1/x_{\text{max},i}^2$ and $R_{jj} = 1/u_{\text{max},j}^2$, where $x_{\text{max},i}$ is the deviation we accept on state i (2 cm, 0.2 m/s, 0.3 rad, 3 rad/s) and $u_{\text{max},j} = 0.5$; then they were tuned by hand. We also add the integral of the position error as three more states (an LQR with integral action, or LQI), otherwise a steady wind leaves a few centimetres of offset. The weakness is the model: the LQR trusts the intact bird, so a sudden 40% wing failure makes it lose control in 96% of the episodes, where the PID always survives.

A bridge to the critics. Because $x^\top P x$ is a true value function, we can compare it with PPO's learned critic (Figure 1). We place the bird at rest at points of the vertical plane through the hover point and evaluate both. Within 5 cm of hover they rank these states in almost the same order (Spearman rank correlation 0.96, averaged over three seeds). Both judge a horizontal error worse than a vertical one: the bird must tilt before it can move sideways, but it can climb at once. Further out (±15 cm, correlation 0.88) they diverge, and the seeds disagree with each other there. The critic is extrapolating beyond the states it saw in training.

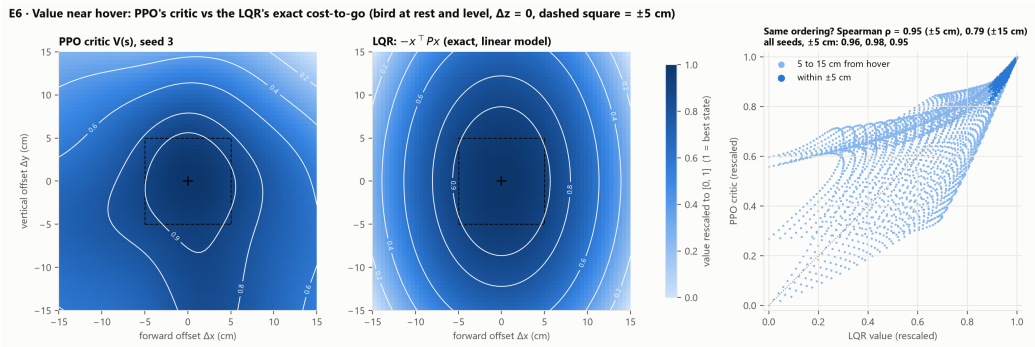


Figure 1: The value of states near hover, for a bird at rest and level at an offset $(\Delta x, \Delta y)$ from its target, in calm air. Left: PPO's learned critic $V_\psi(o)$ (seed 3); middle: the LQR's exact cost-to-go $-x^\top Px$; both rescaled to [0, 1], 1 being the best state, with contour lines. Right: one point per grid cell, LQR value against PPO value; dark points lie within ±5 cm (dashed square), where the two rank states almost identically.

The LQR needs a model and a quadratic cost. Reinforcement learning needs neither: it estimates values like $x^\top Px$ from flights alone, and that is where we turn next.

3. Policy-gradient methods: PPO

From the bird to the networks. The networks never see the simulator's state s_t, only the observation o_t, so from now on we write $\pi_\theta(a \mid o)$, $V_\psi(o)$ and $Q_\psi(o, a)$. Every network is a small multilayer perceptron with two hidden layers of 256 units. The actor maps the 48 numbers of o_t to the 5 actions: PPO's to the means of five Gaussians (plus five learned standard deviations), TD3's to the actions themselves, SAC's to five means and five standard deviations. PPO's critic maps o_t to one number; TD3's and SAC's map (o_t, a_t), 53 numbers, to one number. To give a scale: on Hover a bird that holds the target earns about 1 per decision, so the best possible value is about $1/(1-\gamma) = 100$, while a crash costs -10 at once plus all the reward it forgoes.

Partial observability. Because the wind, the mass and the damage are hidden, $V_\psi(o)$ can only learn the value averaged over the conditions that produce the same o. The policy must choose one action for all of them. A feed-forward network has no memory, so it can be *robust* (an action that works across the conditions seen in training) but not *adaptive* (it cannot estimate the wind over time and correct for it, as the PID's integrator does). The previous action, which is part of o, gives it one step of memory. For example, unequal amplitudes with a level bird can only mean a weaker wing. This is plausibly how the agents trained with damaged wings shrug off a sudden failure: the largest deviation is 1.5 cm for PPO-randomised against 9.1 cm for the PID.

Policy gradient. The policy-gradient theorem (Sutton et al. 2000) gives the direction in which to change θ to increase J:

$$\nabla_\theta J = \mathbb{E}_\pi\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid o_t)\, \hat{A}_t\Big].$$

With the weight $\hat{A}_t$ replaced by the return-to-go $G_t = \sum_{k\geq 0} \gamma^k r_{t+k}$, this is REINFORCE (Williams 1992): unbiased, but very noisy. Subtracting a baseline $b(o_t)$ leaves the expectation unchanged and reduces the variance. The best simple baseline is the value itself, which turns the weight into an *advantage*: how much better the action was than the policy's average. A second network, the critic V_ψ, learns that baseline by regression: this is actor-critic. Since $\log \pi_\theta(a \mid o)$ is a sum over the five action dimensions, one gradient step nudges all five Gaussians at once, towards the actions that did better than expected and away from the others.

GAE (generalised advantage estimation; Schulman et al. 2016). The one-step temporal-difference (TD) error $\delta_t = r_t + \gamma V_\psi(o_{t+1}) - V_\psi(o_t)$ is an advantage estimate with low variance, but it is only as good as the critic. The Monte-Carlo estimate $G_t - V_\psi(o_t)$ is the opposite: unbiased and noisy. GAE interpolates between the two with a parameter $\lambda \in [0, 1]$:

$$\hat{A}_t = \sum_{l\geq 0} (\gamma\lambda)^l\, \delta_{t+l}, \qquad \lambda = 0:\ \hat{A}_t = \delta_t, \qquad \lambda = 1:\ \hat{A}_t = G_t - V_\psi(o_t).$$

CleanRL computes it backwards through the rollout, in two lines (`ppo_continuous_action.py` L244–245). The term `nextnonterminal` zeroes the bootstrap at a termination.

PPO (Schulman et al. 2017). One batch of data is used for several epochs of minibatch updates. To stop the policy from drifting far from the policy that collected the data, PPO clips the probability ratio $\rho_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\theta_{\text{old}}}(a_t \mid o_t)$ to $[1-\epsilon, 1+\epsilon]$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\big[\min\left(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\big].$$

Once an action has become $1 \pm \epsilon$ times more (or less) likely in the direction its advantage favours, its gradient vanishes. For our bird the ratio is the product of five per-action ratios, so the clip bounds the change of the whole action (both wings, the frequency and both tilts) rather than of each one. CleanRL's loss (L280–282) is $\max(-\hat{A}\rho,\ -\hat{A}\,\text{clip}(\rho))$, the same thing with the sign flipped for gradient descent. The full loss adds a value term and an entropy bonus, $L = -L^{\text{CLIP}} + c_v\,(V_\psi(o_t) - \hat{R}_t)^2 - c_e\,\mathcal{H}[\pi_\theta(\cdot \mid o_t)]$ (L300), where $\hat{R}_t = \hat{A}_t + V_{\psi_{\text{old}}}(o_t)$ is the regression target of the critic and c_v, c_e are weights. Advantages are standardised per minibatch (L276–277), gradients are clipped to norm 0.5 (L304), and the learning rate decays linearly to zero (L202–205).

Our settings. The policy is $\mathcal{N}(\mu_\theta(o), \text{diag}\,\sigma^2)$ with a learned σ that does not depend on o, and the actor and critic are separate networks with tanh activations. Each update uses 8 environments $\times$ 512 steps = 4,096 samples, 10 epochs of minibatches of 256, $\epsilon = 0.2$, $\lambda = 0.95$, a learning rate of 3×10^{-4} annealed to 0, $c_v = 0.5$, $c_e = 0$, and rewards divided by a running estimate of the return's standard deviation (`VecNormalize`). SB3 does not clip the value loss by default (`clip_range_vf=None`), unlike CleanRL (`clip_vloss`, L286). At test time the action is the mean, clipped to $[-1, 1]$. Three choices of ours matter on this bird:

- **Small initial noise.** SB3 starts with $\sigma = 1$: the two amplitudes would then differ by about 55° at random, and the roll axis is so responsive (about 2,300 rad/s² of authority) that the bird would flip at once. We start at $\sigma = e^{-1} = 0.37$ (about ±15° per wing).
- **Start at hover.** The untrained network outputs $a \approx 0$, half amplitude, which is not enough lift, so every early episode is a fall. We set the bias of the last layer to the hover action and shrink its weights by 100. The agent then starts from the trim point, as a classical controller would.
- **Clipped mean, not tanh.** Almost all of PPO's few hover failures (2%) are instant overshoots. After a strong push it commands full roll, and the bird passes 80° within 3 to 5 decisions. A tanh-squashed policy, as in SAC and TD3, approaches the limits more gently, and neither of them shows these failures.

PPO is reliable, but it throws each batch away after ten epochs. With a simulator this cheap we can afford it; with real flights we could not. The off-policy methods keep every transition.

4. Off-policy actor-critic methods: TD3 and SAC

The DDPG idea (deep deterministic policy gradient; Lillicrap et al. 2016). Q-learning picks the best action by $\max_a Q(o, a)$, which is impossible to compute exactly over five continuous actions. So an actor $\mu_\theta(o)$ is trained to do it, by gradient ascent on $Q_\psi(o, \mu_\theta(o))$: the critic tells the actor in which direction of the five-dimensional action space the value increases. The critic regresses on a bootstrapped target

$$y = r + \gamma\,(1 - \text{done})\,Q_{\bar{\psi}}(o', \mu_{\bar{\theta}}(o')),$$

where o' is the next observation and done is 1 only if the transition terminated the episode. The target uses *target networks* $\bar{\psi}, \bar{\theta}$, copies that follow the online networks slowly, $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$ with $\tau = 0.005$, so that the target does not chase itself (Mnih et al. 2015). Transitions come from a *replay buffer* of past experience, so every sample is reused many times. This is where the sample efficiency comes from.

TD3 (Fujimoto et al. 2018) fixes DDPG's overestimation of Q with three changes, all visible in `td3_continuous_action.py`:

- **Clipped double Q-learning** (L239–242): two critics ψ_1, ψ_2, and the target uses the smaller one,

$$y = r + \gamma\,(1 - \text{done}) \min_{i=1,2} Q_{\bar{\psi}_i}(o', \tilde{a}').$$

- **Target policy smoothing** (L232–236): $\tilde{a}' = \text{clip}(\mu_{\bar{\theta}}(o') + \text{clip}(\varepsilon, -c, c), -1, 1)$, with $\varepsilon \sim \mathcal{N}(0, 0.2^2)$ and $c = 0.5$, so that the critic cannot exploit a narrow peak.
- **Delayed actor updates** (L255): the actor and the targets move once every two critic updates, so the actor follows a critic that has had time to settle.

Exploration adds $\mathcal{N}(0, 0.1^2)$ noise to each of the five actions, about $\pm 4°$ of wing amplitude. In SB3 these settings are `policy_delay=2`, `target_policy_noise=0.2`, `target_noise_clip=0.5` and `action_noise`.

SAC (Haarnoja et al. 2018a, b) maximises reward *plus entropy*, $J = \mathbb{E}[\sum_t \gamma^t (r_t + \alpha \mathcal{H}[\pi_\theta(\cdot \mid o_t)])]$, where the temperature $\alpha > 0$ sets the price of randomness. The policy keeps acting as randomly as the task allows, which drives exploration and avoids premature commitment. The soft Bellman target uses two critics as in TD3, and pays for the next action's log-probability (L261–262):

$$y = r + \gamma(1 - \text{done})\Big(\min_i Q_{\bar{\psi}_i}(o', a') - \alpha \log \pi_\theta(a' \mid o') \Big), \qquad a' \sim \pi_\theta(\cdot \mid o').$$

The actor is a tanh-squashed Gaussian: $\tilde{a} = \tanh(z)$ with $z = \mu_\theta(o) + \sigma_\theta(o) \odot \xi$, $\xi \sim \mathcal{N}(0, I)$ and $\odot$ the element-wise product. Written this way (the *reparameterisation trick*, L143–145), the sample is a differentiable function of θ, so the actor loss $\mathbb{E}[\alpha \log \pi_\theta(\tilde{a} \mid o) - \min_i Q_{\psi_i}(o, \tilde{a})]$ (L283) can be backpropagated through the critic. The squashing changes the density, hence the correction $-\sum_j \log(1 - \tanh^2 z_j)$ (L148). Finally, the temperature is learned to hold the entropy near a target $\bar{\mathcal{H}} = -5$ (minus the number of actions), by minimising $\mathbb{E}[-\alpha(\log \pi_\theta(\tilde{a} \mid o) + \bar{\mathcal{H}})]$ (L292–297; SB3: `ent_coef="auto"`). On our bird that target corresponds to a noise of roughly 0.09 per action, again a few degrees of amplitude.

Our settings and what we saw. We use one environment, a buffer of 10^6 transitions, 10,000 random steps before learning, one gradient step per environment step, minibatches of 256, learning rates of 3×10^{-4} (SAC) and 10^{-3} (TD3), and ReLU activations. Two observations:

- **Fewer samples, much more compute.** On Obstacles, SAC reaches 90% validation success after 0.45 to 0.65 M steps, against 0.95 to 2.0 M for PPO (one PPO seed of five never does). With 7 jobs on our CPU, though, SAC runs at about 43 steps/s against 700 for PPO: one gradient step per environment step costs more than eight parallel environments and 160 gradient steps per 4,096 samples.
- **Unstable training.** On Hover, all four off-policy configurations (SAC and TD3, trained in normal and in randomised conditions) reach a high validation success early and then swing (Figure 2). SAC's final model succeeds in 64% of the test episodes but its best checkpoint in 99%. For TD3 seed 3 the figures are 7% and 100%. We report the final model, as the protocol says, with the best checkpoint next to it.



Figure 2: The off-policy agents on Hover (constant learning rate). Left: validation success (50 fixed episodes) of every seed during training; SAC (orange) and TD3 (green), dotted when trained in normal conditions and solid when trained in randomised ones; PPO's mean (blue), whose learning rate is annealed, for reference. SAC and TD3 swing from one evaluation to the next, PPO does not. Right: test success (200 episodes) of each SAC and TD3 seed, final model (open circle) against best-validation checkpoint (filled).

Analysis of off-policy stability. We first read this as a late degradation caused by the constant learning rate, the same drift PPO showed in our pilots before we annealed its rate. We tested it: SAC and TD3 retrained with a learning rate that decreases linearly to zero, like PPO's, end at 76% (seeds: 96, 56, 76) and 64% (60, 32, 100) on the test episodes, against 64% and 69% before, while their best checkpoints stay at 96% and 100%. Annealing barely helps. The validation curves show why (Figure 2): the success of SAC and TD3 does not decay, it oscillates throughout

training. SAC seed 1 goes from 100% to 48%, 0%, 94% and 74%; TD3 seed 3 from 100% to 24%, back to 100%, and down to 6% at the very last evaluation. The final model is wherever the oscillation happened to be when training stopped, and the best checkpoint is the top of one swing. Why would the off-policy agents oscillate on this bird, and PPO not? We have hypotheses, not proofs:

1. *A critic trained on easy states.* Once the agent hovers well, almost every new transition is a near-hover one. The critic is fitted more and more to those, and less to the rare, large deviations that start each episode. But on an unstable bird those are the states where a wrong action is fatal. SAC's logged temperature fits this picture: after a dip to 0.006 at the start of learning, the learned α settles around 0.02, so the policy is nearly deterministic and seldom leaves the hover point. With α fixed ten times higher (0.2) the noisier policy keeps visiting perturbed states, and SAC ends at 85%.
2. *An actor that follows the critic's errors.* The actor climbs the critic's gradient $\nabla_a Q$, so any overestimated region pulls the actions towards it, which is the failure TD3's twin critics reduce but do not remove (Fujimoto et al. 2018).
3. *A plant that amplifies small changes.* On an unstable bird a small change of the feedback can turn a well-damped hover into a slow oscillation, and precision goes first. Most failing final models do still fly, but settle off target: 2.4 to 3.2 cm away on average for SAC (just outside the 3 cm rule) and 5 cm for TD3 seed 3. The worst case, TD3-randomised seed 2, flies out of the arena in all 200 test episodes.

The logged critic loss does not rise consistently when the success falls, so we cannot yet tell these apart. PPO is protected on several counts: it learns only from fresh data of the current policy, its steps are clipped, and its learning rate goes to zero. The practical lesson for an off-policy agent on an unstable system is to select the model on validation, as the best checkpoint does, rather than to take the last one. Section 5 now sets the five controllers side by side.

5. Comparison

	PID	LQR	PPO	TD3	SAC
Needs a model	nominal (geometry, allocation)	linearised nominal	no	no	no
Learns from	hand tuning	Riccati equation	on-policy rollouts	replay buffer	replay buffer
Policy at test	deterministic	linear, $u = -Kx$	Gaussian mean, clipped	deterministic	tanh of the mean
Value	none	$x^\top P x$, exact (linear model)	$V_\psi(o)$	twin $Q_\psi(o, a)$	twin soft $Q_\psi(o, a)$
Hover: steps to 90% validation	—	—	0.10–0.15 M	**0.05–0.10 M**	0.10–0.15 M
Steps per second (7 jobs)	—	—	≈ **700**	≈ 86	≈ 43
Hover test success (final / best)	**100%**	**100%**	98%	69% / **100%**	64% / 99%
Robust area, hover / go-to-point (normal training)	**0.82** / **0.81**	0.57 / 0.56	0.39 / 0.36	0.12 / 0.30	0.08 / 0.32
Same, randomised training	—	—	**0.79** / **0.96**	0.39 / 0.93	0.36 / 0.82
Obstacles / Navigation success	44% / **41%**	52% / 28%	89% / 18%	—	**98%** / 10%

The robust area is the mean success over a 5 × 5 grid of wind (0 to 40% of the weight) × wing damage (0 to 40%); for SAC and TD3 on hover it is that of the best checkpoint. Values are means over 3 seeds (5 for PPO on Obstacles and Navigation); the best value of each row is in bold.

What the table says about the algorithms, beyond the scores:

- **A memoryless policy has no integral.** Trained in calm air, every RL agent is fragile (robust area 0.08 to 0.39 against 0.82 for the PID). Half to almost all of their failures on the grid (50 to 96%) are drift: under a steady wind the bird settles a few centimetres off target, like a proportional controller with its steady-state error. This is the partial observability of Section 3 at work: the wind is not in o, and the network cannot integrate the error. Training under randomised conditions (domain randomisation; Tobin et al. 2017) teaches it to compensate from what it does see (tilt, previous action), and on go-to-point all three algorithms then match or beat the PID. A recurrent policy, or the integrated error as an input, would give it the PID's mechanism directly.

- **On- vs off-policy is a trade between samples and compute.** On a cheap simulator running on a CPU, PPO's wall-clock time wins. With an expensive simulator or real flights, SAC's sample efficiency would.
- **Model-based optimal is only optimal for the model.** In calm air the LQR is about as precise as the PID (0.55 cm against 0.44 cm on hover), but it fails along the one axis its model ignores, wing damage.

Section 6 takes the learning algorithms apart, one component at a time.

6. Components and ablations

Each ablation changes one component and keeps everything else as trained (Hover unless stated, 3 seeds, 200 test episodes; Figure 3).

Component	Equation	CleanRL	SB3 setting (ours)	Ablation	Observed
Clipped surrogate	L^{CLIP}	PPO L280–282	`clip_range=0.2`	`clip_range=1000`	98% → **0%**; never learns to stay up
Advantage normalisation	$\hat{A} \leftarrow (\hat{A} - \bar{A})/\sigma_A$	PPO L276–277	`normalize_advantage`	`False`	99%: no effect
Reward normalisation	$r_t \leftarrow r_t/\sigma_R$	PPO L99	`VecNormalize` (ours)	off; off and no advantage normalisation	96%; 96%
GAE	$\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}$	PPO L244–245	`gae_lambda=0.95`	$\lambda = 0$; $\lambda = 1$	**0%** (ends 58 cm away); 94%
Learning-rate annealing (PPO)	$\eta_k = \eta_0(1 - k/K)$	PPO L202–205	linear schedule	constant (pilot, 1 seed)	validation 94% → 80%
Learning-rate annealing (SAC, TD3)	same	—	constant (SB3 default)	annealed	SAC 64% → 76%; TD3 69% → 64%
Start at hover	last-layer bias = hover action	—	— (ours)	default init	98%, but 90% reached after 0.23 M steps, not 0.12 M
Entropy temperature	$\min_\alpha \mathbb{E}[-\alpha(\log\pi + \bar{\mathcal{H}})]$	SAC L292–297	`ent_coef="auto"` (learned $\alpha \approx 0.02$)	`ent_coef=0.2`	64% → 85%
Failure penalty (Navigation)	r_{fail}	—	−10 (ours)	−50	18% → 10%

In the table, $\bar{A}$ and σ_A are the mean and standard deviation of the advantages in a minibatch, σ_R a running standard deviation of the discounted return, η_k the learning rate at update k of K, and r_{fail} the reward of a failure.

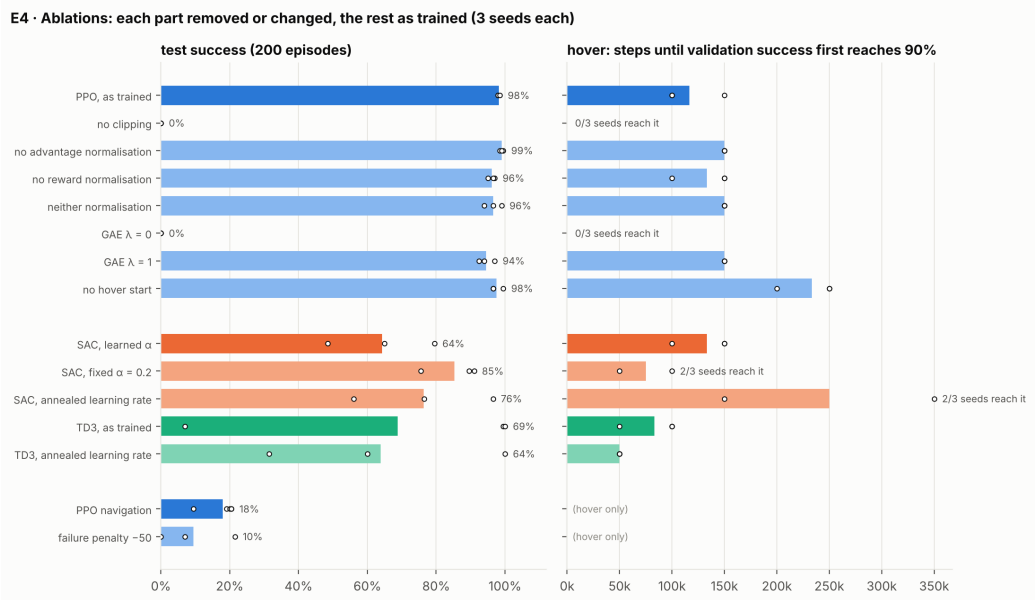


Figure 3: Ablations: each variant changes one component and keeps the rest as trained (3 seeds each, normal conditions). Left: test success over 200 episodes; bars are means over seeds, dots single seeds. Right, Hover only: environment steps until validation success first reaches 90%, averaged over the seeds that reach it.

- **Clipping is essential here.** Without it, ten epochs on the same batch let the policy move far from the data its advantages were estimated on. On an unstable bird one bad update means a crash, the next batch is all crashes, and no seed ever recovers. Its losses of control are spread over the whole episode, unlike the instant overshoots of the full PPO.
- **$\lambda = 0$ learns to survive but not to come back.** The crash penalty is immediate, so one-step advantages see it. The reward for returning to the target arrives only after many decisions and must travel through the critic, and at $\lambda = 0$ the whole credit assignment rests on a critic that is still poor. The bird stays airborne, 40 to 80 cm from the target. With $\lambda = 1$ the advantages are unbiased but noisy: 94%, with more losses of control. The default 0.95 sits between the two.
- **Advantage normalisation does nothing here, and not for the reason we first thought.** Our guess was that the rewards, already normalised by `VecNormalize`, made it redundant. We tested it: with the reward normalisation switched off too, removing advantage normalisation still changes nothing (96% with it, 96% without). The likelier reason is the optimiser. Adam (Kingma & Ba 2015) divides each weight's update by the running size of that weight's own gradients, so multiplying every advantage by a constant leaves the actor's updates almost unchanged, and our actor and critic are separate networks. What normalisation still does, subtracting the mean, matters little once the critic is good. Reward normalisation itself helps a little (98% and 0.78 cm with it, 96% and 0.99 cm without). This is a reminder that "implementation details" interact (Engstrom et al. 2020; Andrychowicz et al. 2021; Huang et al. 2022b).
- **The hover start halves the time to learn and changes nothing at the end.** It is a prior, not a different solution.
- **A fixed temperature helped SAC**, from 64% to 85% at the end of training, but not for the reason we first guessed. The learned temperature does not shrink when the success drops: after a dip to 0.006 at the start of learning it stays mostly between 0.01 and 0.03 for the rest of training. The fixed value, 0.2, is simply ten times larger, so the policy stays noisier, which plausibly keeps more off-hover states in the replay buffer (Section 4).
- **Annealing the learning rate is not the cure for SAC and TD3.** It fixed PPO's late drift in our pilots, but SAC and TD3 trained with it still swing (Section 4): 76% and 64% at the end, 96% and 100% at their best checkpoints.
- **A harsher failure penalty did not make navigation safer.** Collisions stayed at 66 to 84%. A collision penalty is a *sparse* signal: it arrives once, at the moment of contact, and says nothing about the decisions that led there. Making it larger only makes that single signal louder. A *dense* term, a small penalty that grows as the bird comes closer to an obstacle, would tell it at every decision how close it is to danger. Written as the difference of a potential between two states, such a term does not even change which policy is optimal (Ng et al. 1999). We have not trained with it yet.

7. Limitations and what we would test next

- **Seeds.** Three seeds per configuration (five for some) show the spread but give wide intervals; the transfer gain on Navigation, for instance, is suggestive rather than conclusive (paired t-test $p = 0.16$).
- **Why the off-policy agents oscillate remains open.** We ruled out the constant learning rate and a shrinking temperature (Sections 4 and 6). The remaining hypotheses need direct tests: a replay buffer that keeps the hard starts (prioritised or stratified replay), a slowly averaged copy of the actor used for evaluation, or larger validation sets to select the model.
- **Memory.** Our policies are feed-forward. A recurrent policy, or the integrated position error as an extra input, would test directly whether memory closes the gap with the PID under steady wind.
- **Reward.** Obstacles are only penalised on contact; a dense proximity term is the natural next step.
- **E6 covers one plane.** The value landscape and the vector fields are measured in the vertical plane through the hover point, with the bird at rest; other slices of the 48-dimensional observation could look different.
- **The simulator.** The physics averages the wings over each wingbeat and models wind as a force on the body, so the controllers never meet the unsteady aerodynamics of a real hummingbird.

References

- Andrychowicz M. et al. (2021). What matters in on-policy reinforcement learning? A large-scale empirical study. *ICLR*.
- Bryson A. E., Ho Y.-C. (1975). *Applied Optimal Control*. Hemisphere.
- Engstrom L. et al. (2020). Implementation matters in deep policy gradients: a case study on PPO and TRPO. *ICLR*.
- Fujimoto S. et al. (2018). Addressing function approximation error in actor-critic methods. *ICML*.

- Haarnoja T. et al. (2018a). Soft Actor-Critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML*.
- Haarnoja T. et al. (2018b). Soft Actor-Critic algorithms and applications. arXiv:1812.05905.
- Huang S. et al. (2022a). CleanRL: high-quality single-file implementations of deep reinforcement learning algorithms. *JMLR* 23(274). Line numbers refer to commit `fe8d8a0` (April 2026).
- Huang S. et al. (2022b). The 37 implementation details of Proximal Policy Optimization. *ICLR Blog Track*.
- Kingma D. P., Ba J. (2015). Adam: a method for stochastic optimization. *ICLR*.
- Khatib O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. *Int. J. Robotics Research* 5(1).
- Koren Y., Borenstein J. (1991). Potential field methods and their inherent limitations for mobile robot navigation. *ICRA*.
- Lillicrap T. et al. (2016). Continuous control with deep reinforcement learning. *ICLR*.
- Mnih V. et al. (2015). Human-level control through deep reinforcement learning. *Nature* 518.
- Ng A. Y. et al. (1999). Policy invariance under reward transformations: theory and application to reward shaping. *ICML*.
- Pardo F. et al. (2018). Time limits in reinforcement learning. *ICML*.
- Raffin A. et al. (2021). Stable-Baselines3: reliable reinforcement learning implementations. *JMLR* 22(268).
- Schulman J. et al. (2016). High-dimensional continuous control using generalized advantage estimation. *ICLR*.
- Schulman J. et al. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.
- Sutton R. S., Barto A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press.
- Sutton R. S. et al. (2000). Policy gradient methods for reinforcement learning with function approximation. *NeurIPS* 12.
- Tobin J. et al. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. *IROS*.
- Williams R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8.

How this project was built. Hummingbirds fascinate me: they are beautiful, their flight is both graceful and remarkably precise, and they are the only birds that can hover for long, fly backwards and turn on the spot. I wanted to learn reinforcement learning on a problem I cared about. I defined the tasks, the experiments and the comparisons, chose between the options at each step, ran every training and evaluation on my own computer and interpreted the results. I used Claude (Anthropic) to write most of the code (the simulator, the training and evaluation pipeline, the web demo) and to draft the text of both reports from my results, which I reviewed, questioned and corrected. I studied the simulator, the tasks, the classical controllers and the learning algorithms closely, and the training setup, the configurations, the evaluation suites, the analysis and the tests in less depth. The rest (the export to the browser, the scripts and the web demo) I checked from the outside: what each part takes in, what it returns and why.

A first attempt. This is my second version of the project. During the summer holidays I spent three weeks on a first hummingbird simulation, also built with AI assistance, set in a much more complex world and with a large part written in C++: a physics engine in C++, linked to Python for the learning (I got this bridge between the two languages working), and three 3D viewers, about 15,000 lines of C++ in all. It grew too large to finish, and reviewing it showed me why. The pieces never came together, some tasks did not test what they claimed (a 30-line hand-written controller already solved most of one), and the C++ engine was not the bottleneck: physics took only about 28% of the training time. So I started again, smaller, in Python, with classical baselines from the start.

What I learned. Some flight physics, at the level of a simplified model: how the wingbeat frequency, the amplitude of each wing and the tilt of the stroke planes produce the lift and the rolling, pitching and yawing torques that steer the bird, and why hovering is unstable. How a PID and an LQR stabilise an unstable system, and why the LQR is optimal only for its own model. How PPO, TD3 and SAC work, part by part, and which of their components and hyperparameters matter: on this bird, clipping and the GAE λ decide between success and failure, advantage normalisation changes nothing, and the off-policy agents must be chosen on validation. And how to compare learned and classical controllers fairly: the same held-out episodes, several seeds, intervals rather than single numbers.

What I would do differently. Even in its second version, the project combines flight physics, fluid dynamics, rigid-body mechanics, control theory, reinforcement learning, statistics, numerical methods and a large codebase with a 3D web application: too much to master alone at this stage. Starting with a simpler problem would have been the better way to learn RL. But it taught me how much scoping matters, and that a good scope needs a real understanding of the problem and of the tools it will use, which I only gained by trying. My next project is to implement PPO from scratch.